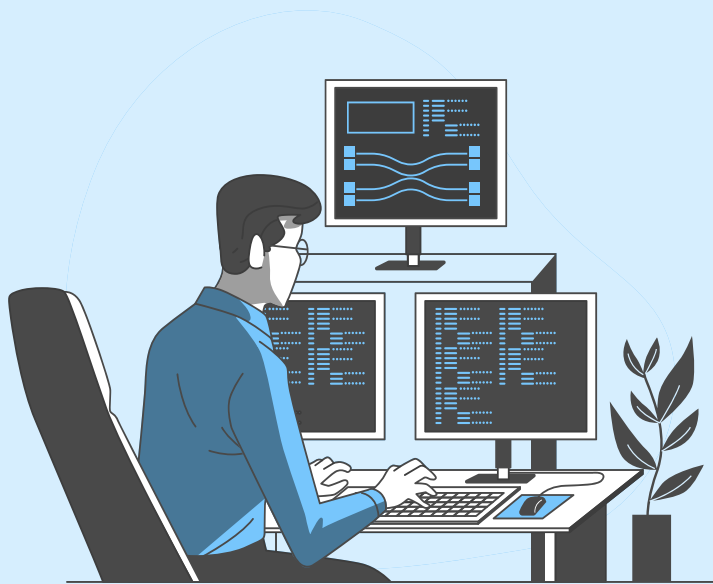


Beyond Manual Tags: Enhancing User Experience with AI-Powered Item Classification

Applied Machine Learning



Cedric KOH Bo Xiang
GAN Seng Zhan
GOH Yun Si
HUANG Mochen
WU Junqi

Agenda

01

Business Problem, Solutions & Pipeline

Business Background, Technical Solutions, Project Pipeline

02

Data Source, EDA and Processing

Data from Kaggle and online scraping
EDA, Processing

03

Modelling

RF, KNN, XGBoost, CNN

04

Conclusion

Evaluation

05

Limitations and Future Work

Limitations, Prospects



Business Statement & Technical Solutions

Business Problem

- SautyFash, the popular second-hand sneaks trading platform
- Two Recurring Feedbacks
 - Tricky Posting Flow
 - Irrelevant Items Recommended

Nike Dunk



Nike Air Force 1 Low

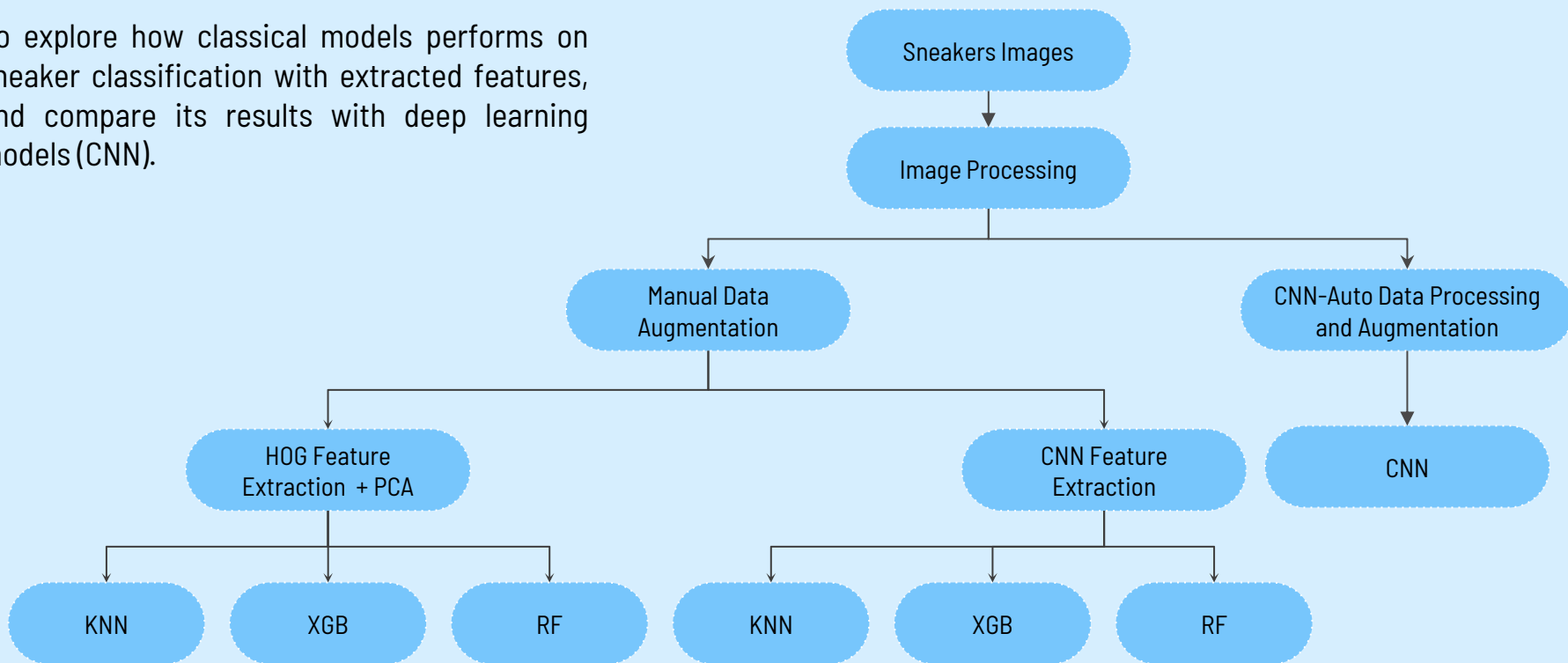


Technical Solutions

- Image Classification
 - Image Processing
 - Machine Learning
 - Data Augmentation
 - HOG Feature Extraction
 - CNN Feature Extraction
 - Modelling
 - Random Forest
 - KNN
 - XGBoost
 - Deep Learning
 - CNN
 - ResNet50

Pipeline

To explore how classical models performs on sneaker classification with extracted features, and compare its results with deep learning models(CNN).



Data Briefing



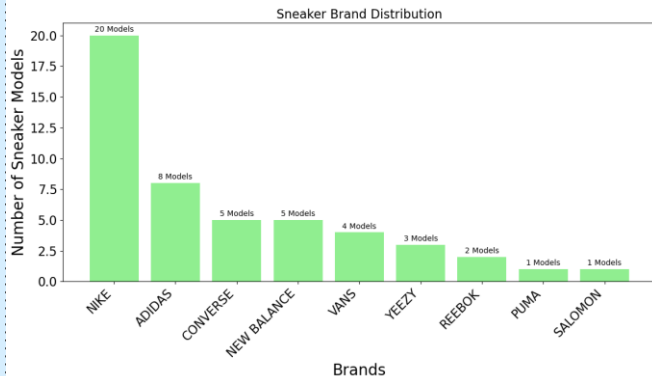
Main Data Set

Popular Sneakers Classification Dataset: <https://www.kaggle.com/datasets/nikolasgegenava/sneakers-classification>

Supplementary Data Set

Scraping Online (batch scraping with IP rotation & random delays)

Brands Summary



Brands:

- Popular Brands: Nike, Adidas, New Balance, Vans, Salomon, Yeezy ...
- 50 Popular Sneaker Models: Air Force 1, Dunk, Air Jordan 1

Sneaker Summary



Image Classes:

- Max #Samples: 150
- Min #Samples: 91
- Mean #Samples: 130
- Median #Samples: 115

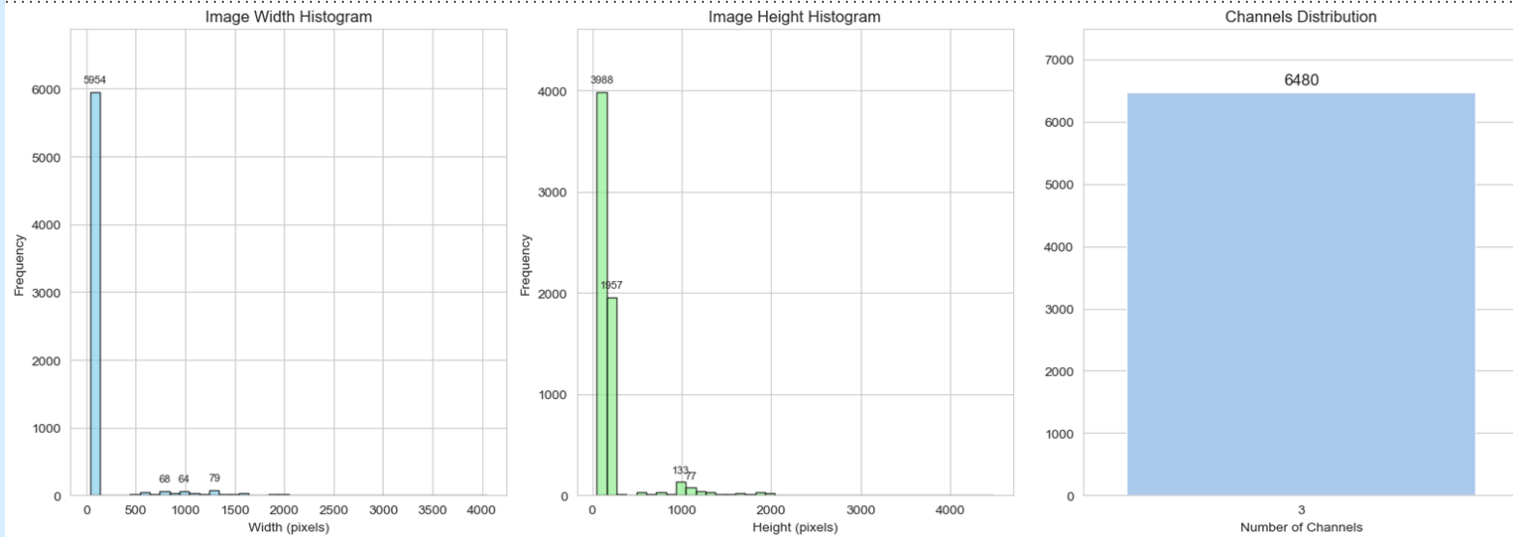
Exploratory Data Analysis, EDA: Image Attribute Analysis

Summary

- Checked image dimensions
 - Width and height distributions
 - Most images within 120–140 pixels height.
 - Decided to resize to 224×224 for consistency.
- Verified all images are RGB, 3 channels per image



Width X Height X Channel



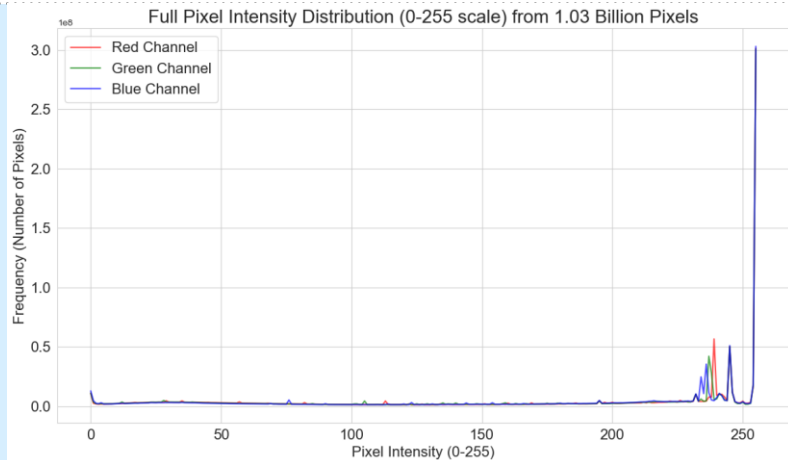
Exploratory Data Analysis, EDA: Pixel Analysis

Summary

- Analysed pixel intensity distribution across all images
- Observed high concentration near maximum brightness
- One peak: very bright
- Confirmed consistent RGB channel behavior



Full Pixel Intensity



Normalised Pixel Intensity

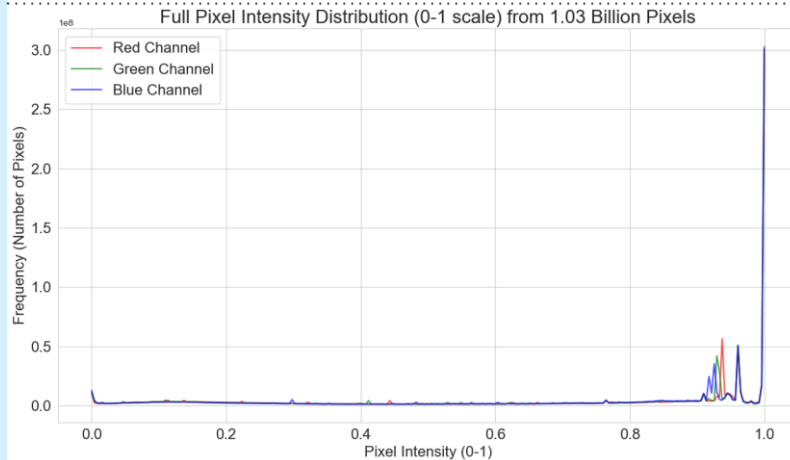
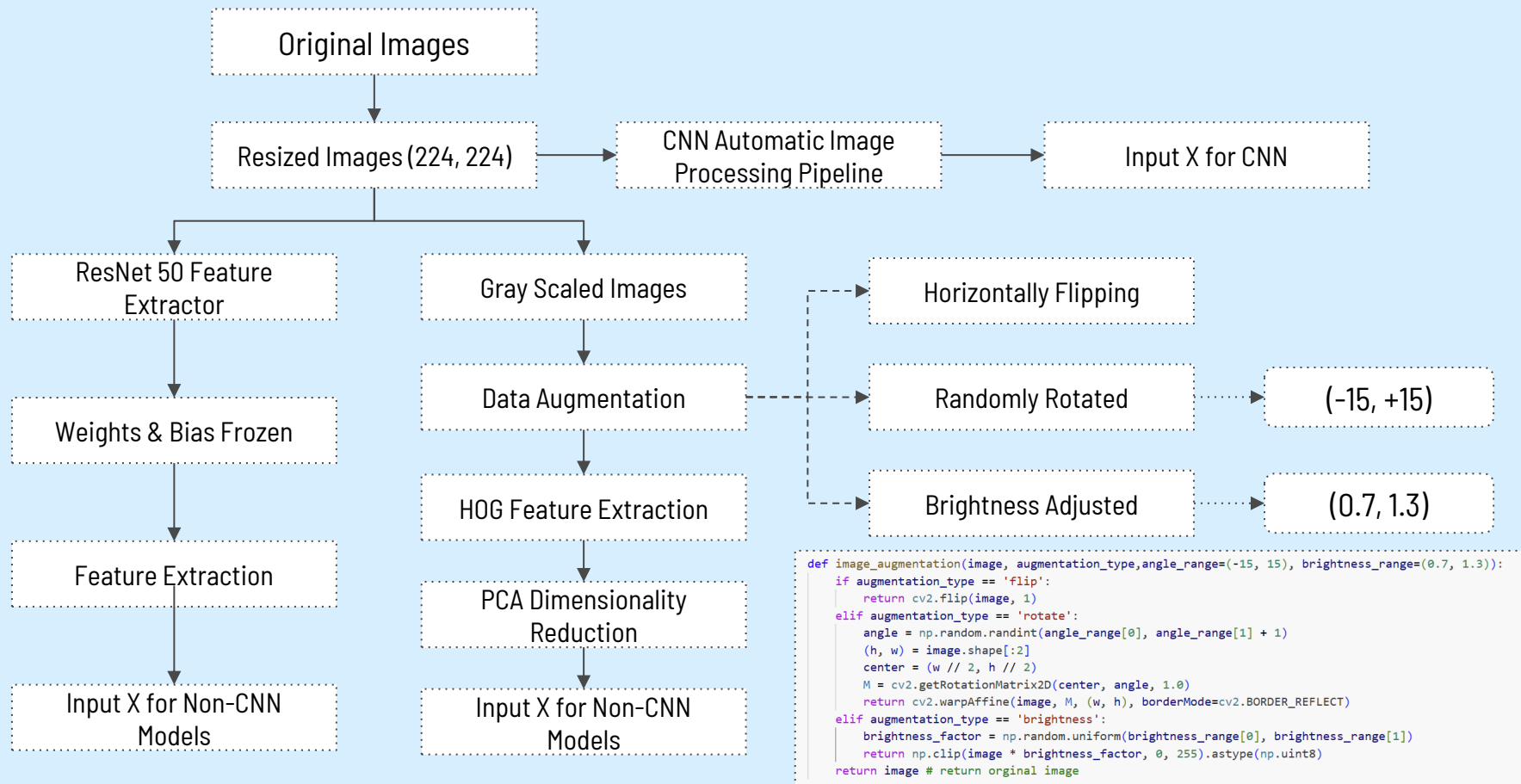


Image Processing



Modelling: Random Forest

Reasoning to use this model: A traditional identification method used before the advent of deep learning, it stands out for its ability to handle high-dimensional data, its robustness against overfitting, and its efficiency through parallelism.

Risks of using this model:

- Replying on manually feature extraction which has limited feature representation capability
- Computationally intensive and memory-hungry when dealing with large datasets or features
- May still overfit in high-dimensional and non-diverse data

HOG Feature Extraction				
	Metric	Average Type	Train	Test
0	Accuracy	N/A	0.999035	0.307099
1	Precision	macro	0.998950	0.328319
2	Recall	macro	0.998908	0.301281
3	F0.5-Score	macro	0.998939	0.313624

CNN Feature Extraction				
	Metric	Average Type	Train	Test
0	Accuracy	N/A	0.998071	0.368827
1	Precision	macro	0.997978	0.391356
2	Recall	macro	0.997817	0.357844
3	F0.5-Score	macro	0.997926	0.356892

HOG VS CNN Extraction

- Both not satisfying
- CNN Extraction Slightly Better

Randomized Search Hyperparameter Tuning

(cv=3, n_iter=10):

- **n_estimators** (50 to 200): more trees generalizes better but slower
- **max_depth** (10 to 40): larger depth fit better but may overfit
- **max_features** (sqrt, log2, 0.5 to 1.0): lower values uses smaller fraction of features, reducing overfitting by adding randomness

Modelling: XGBoost

Reasoning to use this model:

- Robustness to Overfitting
- Flexibility and Hyperparameter Tuning
- Handles Imbalanced Data

Risks of using this model:

- Heavily relies on hyperparameter tuning and prone to overfitting if not properly tuned
- Sensitive to noise
- High computation cost

HOG Feature Extraction				
	Metric	Average Type	Train	Test
0	Accuracy	N/A	0.999035	0.341821
1	Precision	macro	0.998918	0.353889
2	Recall	macro	0.998934	0.337784
3	F0.5-Score	macro	0.998919	0.345981
CNN Feature Extraction				
	Metric	Average Type	Train	Test
0	Accuracy	N/A	0.997492	0.484568
1	Precision	macro	0.997335	0.486872
2	Recall	macro	0.997212	0.479769
3	F0.5-Score	macro	0.997285	0.480678

Randomized Search Hyperparameter Tuning (cv=3, n_iter=10):

1. **n_estimators** (50 to 500): more trees fit better but may overfit
2. **max_depth** (3 to 12): larger depth fit better but may overfit (especially with boosting)
3. **learning_rate** (0.01 to 0.2): lower rate generalizes better but slower and requires more trees
4. **subsample** (0.7 to 1.0): lower values uses smaller fraction of training data, reducing overfitting by adding randomness
5. **colsample_bytree** (0.7 to 1.0): lower values uses smaller fraction of features, reducing overfitting by adding randomness

HOG VS CNN Extraction

- Both not satisfying
- CNN Feature Extraction Much better

Modelling: K-Nearest Neighbour Classifier

Reasoning to use this model:

- “Lazy Learner”: Simplicity and Interpretability

Risks of using this model:

- Long inference time and computationally and memory expensive for large datasets
- Sensitive to noise, class imbalance, and curse of dimensionality
- Manual feature extraction needed, it cannot automatically learn hierarchical, robust, and abstract features directly from raw image pixels.

HOG Feature Extraction				
	Metric	Average Type	Train	Test
0	Accuracy	N/A	0.998939	0.368827
1	Precision	macro	0.998861	0.379593
2	Recall	macro	0.998801	0.367517
3	F0.5-Score	macro	0.998841	0.371319
CNN Feature Extraction				
	Metric	Average Type	Train	Test
0	Accuracy	N/A	0.998071	0.367284
1	Precision	macro	0.997988	0.389494
2	Recall	macro	0.997819	0.361929
3	F0.5-Score	macro	0.997929	0.377110

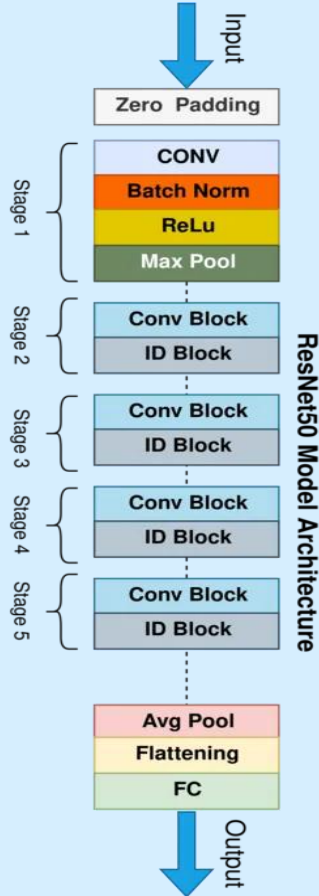
Randomized Search Hyperparameter Tuning (cv=3, n_iter=10):

1. **n_neighbors** (1 to 30): smaller captures local pattern, larger generalizes better
2. **weights** (uniform, distance): equal weights, or higher weights for closer neighbours
3. **metric** (euclidean, manhattan, cosine): cosine calculates angles between vectors (expected to be better for image)

HOG VS CNN Extraction

- Both not satisfying
- The performances for Two methods are similar

Modelling: CNN – ResNet50



Model Utilized: Residual Networks 50 Model (ResNet50)(50 layers)

Reasoning to use pre-trained model - trained on larger dataset so model should be able to identify features better

- Model commonly used for many computer vision tasks^[1]
- Avoid Vanishing Gradient Problem → by Residual Connection^[2]

Methodology:

- 1) Initialise DataLoader for Train, Validation and Test Data
 - Data transformation done differently for Training and Validation/Test dataset
- 1) Initialise and freeze weights for all layers of pretrained model
- 2) Replace the Fully Connected Layer → Transfer Learning
- 3) Epoch training and Tuning by validation
- 4) Model Evaluation using test data

Evaluation Metrics				
Dataset	Accuracy	Precision (Macro)	Recall (Macro)	F-0.5-Score (Macro)
Training	0.9843	0.9843	0.9842	0.9841
Validation	0.8194	0.8335	0.8183	0.8247
Testing	0.7901	0.8087	0.7859	0.7969

Evaluations

Model	Training Time (Minutes)	Accuracy		Precision		Recall		F0.5	
		Train	Test	Train	Test	Train	Test	Train	Test
HOG_KNN	0.0	0.9989	0.3688	0.9989	0.3796	0.9988	0.3675	0.9988	0.3713
HOG_XGB	48.2	0.9990	0.3418	0.9989	0.3539	0.9989	0.3378	0.9989	0.3460
HOG_RF	26.8	0.9990	0.3071	0.9990	0.3283	0.9989	0.3013	0.9989	0.3136
CNN_KNN	0.2	0.9981	0.3673	0.9980	0.3895	0.9978	0.3619	0.9979	0.3771
CNN_XGB	216.4	0.9975	0.4846	0.9973	0.4869	0.9972	0.4798	0.9973	0.4807
CNN_RF	49.2	0.9981	0.3688	0.9980	0.3914	0.9978	0.3578	0.9979	0.3569

Model	Training Time (Minutes)	Accuracy			Precision			Recall			F0.5		
		Train	Valid	Test	Train	Valid	Test	Train	Valid	Test	Train	Valid	Test
CNN	10.5	0.9843	0.8194	0.7901	0.9843	0.8335	0.8087	0.9842	0.8183	0.7859	0.9841	0.8247	0.7969

Conclusions

1 For Feature Extraction:

CNN Feature Extraction > HOG Feature Extraction

2 For Overall Model Performance

Convolutional NN > XGBoost > Random Forest > K-Nearest Neighbour

Limitations & Future Work

Limitations

- **Limited sneaker model coverage:** the current model is limited to specific sneaker brands and models included in the dataset, which may lead to the model may not accurately classify new or niche sneakers
- **Limited data augmentation:** the current data augmentation should be included more operations, and when introducing more operations, the order between gray scaled and data augmentation should be changed.
- **Sensitive to image capture angles:** the model may not function correctly if users take photos from angles that deviate from the requirements, which restricts the flexibility and may lead to classification inaccuracies or failures.
- **Limited training resources:** due to the large number of training data and limited computing resources, hyperparameter tuning was not very comprehensive.
- **Overfitting Problem:** The performance gap between training and testing is around 20%, indicating that the model is overfitted.

Future Work

- Enlarge the datasets to improve accuracy by continuous data collection and diversified data sources
- Train the models on more types and more angles of sneakers to improve the generalisability
- Apply more data augmentation techniques and adjust the order of the pipeline
- Add new capabilities to models such as check authenticity of shoes, resale price prediction
- Future versions can integrate with Large Language Models (e.g. GPT)